

TrendSurfer Pro — Versione Minimal (Pine Script v6)

Strategia e filtri identici all'originale — cambia solo la visualizzazione sul grafico: nessuna tabella, linea, box o etichetta.

▲ Triangolo verde
▼ Triangolo rosso
× Crocetta verde
× Crocetta rossa

Punto d'ingresso LONG (segnale Up Triangle originale, tutti i filtri mantenuti)
Punto d'ingresso SHORT (segnale Down Triangle originale, tutti i filtri mantenuti)
Punto d'uscita dei LONG (Take Profit System bullish)
Punto d'uscita degli SHORT (Take Profit System bearish)

```
1  //@version=6
2  indicator(title = 'TrendSurfer Pro Minimal , overlay = true)
3
4  // =====
5  // TRENDSURFER PRO - VERSIONE MINIMAL
6  // Stessa strategia e stessi filtri dell'originale, cambia SOLO la
7  // visualizzazione. Sul grafico compaiono esclusivamente:
8  // ▲ verde = punto d'ingresso LONG (segnale Up Triangle originale)
9  // ▼ rosso = punto d'ingresso SHORT (segnale Down Triangle originale)
10 // × verde = punto d'uscita dei LONG (Take Profit System bullish)
11 // × rosso = punto d'uscita degli SHORT (Take Profit System bearish)
12 // Nessuna tabella, nessuna linea, nessuna etichetta, nessun box.
13 // Tutta la logica interna (filtri orari, filtri master, filtro volume,
14 // Volatility Score, Sweep Master Filter, ATR Mode) è invariata.
15 // =====
16
17 // === TIMEZONE SETTINGS ===
18 useUTC = input.bool(true, "Use UTC Timezone , group = "Timezone")
19 utcOffset = input.int(2, "UTC Offset (hours)", minval = -12, maxval = 14, group = "Timezone", tooltip = "Set your timezone offset from UTC. Auto-
adjusts when changing assets - Imposta l'offset del fuso orario rispetto all'UTC.")
20
21 // === SIGNALS FILTERS ===
22 enableVWAPFilter = input.bool(false, title = 'Enable VWAP Master Filter for all signals , group = "Signals Filters", tooltip = "Show only
bullish signals above VWAP and bearish signals below VWAP")
23 enableEMA750MasterFilter = input.bool(true, title = 'Enable EMA750 Master Filter for all signals , group = "Signals Filters", tooltip = "Show
only bullish signals above EMA750 and bearish signals below EMA750")
24 enableButterworthMasterFilter = input.bool(false, title = 'Enable Butterworth Master Filter for all signals , group = "Signals Filters", tooltip
= "Show only bullish signals above BF and bearish signals below BF")
25 enableSweepMasterFilter = input.bool(false, title = 'Enable Liquidity Sweep Master Filter for all signals , group = "Signals Filters", tooltip =
"Show signals only within X candles after a Liquidity Sweep")
26 sweepMasterFilterBars = input.int(5, title = 'Max bars after Sweep', minval = 1, maxval = 100, group = "Signals Filters")
27
28 // === ATR VOLATILITY SCORE ===
29 atrVolPeriod = input.int(14, "ATR Period (Volatility Score)", minval = 1, group = "ATR Volatility Score )
30 atrVolMAPeriod = input.int(5, "ATR MA Period (score calibration)", minval = 1, group = "ATR Volatility Score , tooltip = "Moving average of the
ATR used as a reference. ATR > MA = high volatility, ATR < MA = low volatility")
31 atrVolSensitivity = input.float(2.0, "Sensitivity (1-5)", minval = 0.5, maxval = 5.0, step = 0.5, group = "ATR Volatility Score , tooltip =
"Amplifies or reduces the score's response. Higher values = more responsive")
32 enableVolScoreFilter = input.bool(false, "Enable Volatility Score Master Filter for all signals , group = "ATR Volatility Score , tooltip =
"Show signals only if ATR Volatility Score >= minimum threshold")
33 volScoreMinThreshold = input.int(80, "Min Volatility Score (1-100)", minval = 1, maxval = 100, group = "ATR Volatility Score )
34
35 atrVol = ta.atr(atrVolPeriod)
36 atrVolMAAdapted =
37   timeframe.in_seconds() <= 60 ? atrVolMAPeriod * 5 :
38   timeframe.in_seconds() <= 120 ? atrVolMAPeriod * 3 :
39   timeframe.in_seconds() <= 180 ? atrVolMAPeriod * 2 :
40   timeframe.in_seconds() <= 300 ? atrVolMAPeriod :
41   timeframe.in_seconds() <= 900 ? math.round(atrVolMAPeriod / 2) :
42   timeframe.in_seconds() <= 1800 ? math.round(atrVolMAPeriod / 3) :
43   timeframe.in_seconds() <= 3600 ? math.round(atrVolMAPeriod / 5) :
44   timeframe.in_seconds() <= 14400 ? math.round(atrVolMAPeriod * 3) :
45   atrVolMAPeriod
46 atrVolMA = ta.sma(atrVol, math.max(2, atrVolMAAdapted))
47 atrVolDev = atrVolMA > 0 ? (atrVol / atrVolMA - 1.0) * 100.0 * atrVolSensitivity : 0.0
48 atrVolScore = math.max(1, math.min(100, math.round(50 + atrVolDev)))
49
50 // === ATR MODE ===
51 enableATRMode = input.bool(false, title = 'Enable ATR Mode , group = "ATR Bands Filter", tooltip = "When enabled, signals are filtered: only
signals within the ATR channel around EMA750 are shown.")
52 atrMultiplier = input.float(5, title = 'ATR Channel Multiplier (1-100)', minval = 1, maxval = 100, step = 0.5, group = "ATR Bands Filter")
53
54 // === LIQUIDITY SWEEP (usato dal Liquidity Sweep Master Filter) ===
55 enableLiqSweep = input.bool(true, "Enable Liquidity Sweeps , group = "Liquidity Sweep )
56 sweepLookback_TS = input.int(1, "Sweep Lookback (re-entry candles)", group = "Liquidity Sweep , minval = 1, maxval = 10, tooltip = "Maximum
number of candles after a spike beyond a swing level to confirm price has re-entered the range.")
57 swingLenSweep_TS = input.int(5, "Swing Length - Sweep", group = "Liquidity Sweep , minval = 1, maxval = 50, tooltip = "Number of candles left &
right used to identify swing highs/lows for Liquidity Sweep detection.")
58 sweepEMAFilter_TS = input.bool(true, "Apply EMA750 Filter to Sweeps", group = "Liquidity Sweep , tooltip = "Bullish sweeps only above EMA750,
bearish only below")
59 sweepVIDYAFilter_TS = input.bool(false, "Apply VIDYA Filter to Sweeps", group = "Liquidity Sweep , tooltip = "Bullish sweeps only above VIDYA,
bearish only below")
60 sweepVWAPFilter_TS = input.bool(false, "Apply VWAP Filter to Sweeps", group = "Liquidity Sweep )
61
62 // === TOGGLE PER UP/DOWN SIGNAL (INGRESSI) ===
63 enableUpSignal = input.bool(true, title = 'Enable Long Entry Signal ▲', group = "Entry Signals Settings ▲▼", tooltip = "Mostra/Nascondi i punti
d'ingresso long (triangolo verde)")
64 enableDownSignal = input.bool(true, title = 'Enable Short Entry Signal ▼', group = "Entry Signals Settings ▲▼", tooltip = "Mostra/Nascondi i punti
d'ingresso short (triangolo rosso)")
65 src = close
66 sm = input.int(0, title = 'Smoothing Period', group = "Entry Signals Settings ▲▼")
```

```

67 cd = input.float(1, title = 'Constant D', group = "Entry Signals Settings ▲▼")
68
69 // === FILTRO VOLUME PER SEGNALI D'INGRESSO ===
70 enableVolumeFilter = input.bool(true, title = 'Entry Signals Volume Filter 📊', group = "Entry Signals Volume Filter ▲ / ▼")
71 volumeFilterMode = input.string("Range", title = "Volume Filter Mode", options = ["Singolo", "Range"], group = "Entry Signals Volume Filter ▲ / ▼", tooltip = "Show only signals with a FIXED volume value VS a value within the RANGE")
72 volumeFilterSingle = input.int(10, title = "Specific Volume (6-10)", minval = 6, maxval = 10, group = "Entry Signals Volume Filter ▲ / ▼")
73 volumeFilterMin = input.int(6, title = "Min Volume (6-10)", minval = 6, maxval = 10, group = "Entry Signals Volume Filter ▲ / ▼")
74 volumeFilterMax = input.int(10, title = "Max Volume (6-10)", minval = 6, maxval = 10, group = "Entry Signals Volume Filter ▲ / ▼")
75
76 // === TAKE PROFIT SYSTEM (USCITE) ===
77 enableTakeProfit = input.bool(true, "Enable Exit Signals (Take Profit System) ☺", group = "Exit Signals xx")
78 applyTPTimeFilter = input.bool(true, "Apply Time Filters to Exit signals", group = "Exit Signals xx")
79 applyTPVWAPFilter = input.bool(false, "Apply VWAP Filter to Exit signals", group = "Exit Signals xx")
80 // TP System - parametri fissi (identici all'originale)
81 useTPRSI = true
82 useTPMACD = false
83 useTPStoch = true
84 useTPBB = false
85 useTPSTrend = true
86 useTPCCI = true
87 useTPEMA = false
88 useTPPriceAction = true
89 minTPConfirmations = 5
90 tpRsiLen = 25
91 tpRsiOB = 55
92 tpRsiOS = 45
93 tpMacdFastLen = 12
94 tpMacdSlowLen = 26
95 tpMacdSignalLen = 9
96 tpStochKLen = 14
97 tpStochDLen = 3
98 tpStochSmoothK = 3
99 tpBBLen = 20
100 tpBBMult = 2.0
101 tpSTAttrMult = 3.0
102 tpSTAttrLen = 10
103 tpCCIlen = 20
104 tpCCI0B = 100
105 tpCCI0S = -100
106 tpEMAFastLen = 12
107 tpEMASlowLen = 26
108 tpLookBackBars = 5
109
110 // === FILTRO ORARIO 1 ===
111 enableTimeFilter = input.bool(true, title = 'Enable Time Filter 1 ☺', group = "Time Filter 1")
112 timeFilter1Session = input.session("0600-1400", title = 'Time Filter 1 Range', group = "Time Filter 1", tooltip = "Session format automatically adjusts to asset timezone")
113 // === FILTRO ORARIO 2 ===
114 enableTimeFilter2 = input.bool(true, title = 'Enable Time Filter 2 ☺', group = "Time Filter 2")
115 timeFilter2Session = input.session("1400-2200", title = 'Time Filter 2 Range', group = "Time Filter 2", tooltip = "Session format automatically adjusts to asset timezone")
116 // === FILTRO ORARIO 3 ===
117 enableTimeFilter3 = input.bool(true, title = 'Enable Time Filter 3 ☺', group = "Time Filter 3")
118 timeFilter3Session = input.session("2200-0600", title = 'Time Filter 3 Range', group = "Time Filter 3", tooltip = "Session format automatically adjusts to asset timezone")
119
120 // === ALERTS SETTINGS ===
121 enableEntryLongAlert = input.bool(true, title = 'Enable Long Entry Alert ▲', group = "Alerts Settings 📢")
122 enableEntryShortAlert = input.bool(true, title = 'Enable Short Entry Alert ▼', group = "Alerts Settings 📢")
123 enableExitLongAlert = input.bool(true, title = 'Enable Long Exit Alert x', group = "Alerts Settings 📢")
124 enableExitShortAlert = input.bool(true, title = 'Enable Short Exit Alert x', group = "Alerts Settings 📢")
125
126 // =====
127 // FUNZIONI
128 // =====
129 getTimezoneString() =>
130     useUTC ? "UTC" + (utcOffset >= 0 ? "+" + str.toString(utcOffset) : str.toString(utcOffset)) : ""
131
132 passesVolScoreFilter() =>
133     not enableVolScoreFilter or atrVolScore >= volScoreMinThreshold
134
135 vidya_calc(src_vidya, vidya_length, vidya_momentum) =>
136     float momentum = ta.change(src_vidya)
137     float sum_pos_momentum = math.sum(momentum >= 0 ? momentum : 0.0, vidya_momentum)
138     float sum_neg_momentum = math.sum(momentum >= 0 ? 0.0 : -momentum, vidya_momentum)
139     float abs_cmo = math.abs(100 * (sum_pos_momentum - sum_neg_momentum) / (sum_pos_momentum + sum_neg_momentum))
140     float alpha = 2 / (vidya_length + 1)
141     var float vidya_value = 0.0
142     vidya_value := alpha * abs_cmo / 100 * src_vidya + (1 - alpha * abs_cmo / 100) * nz(vidya_value[1])
143     ta.sma(vidya_value, 15)
144
145 // === TIME FILTER ===
146 isInTimeRange() =>
147     if not enableTimeFilter and not enableTimeFilter2 and not enableTimeFilter3
148         true
149     else
150         inRange1 = enableTimeFilter ? not na(time(timeframe.period, timeFilter1Session, getTimezoneString())) : false
151         inRange2 = enableTimeFilter2 ? not na(time(timeframe.period, timeFilter2Session, getTimezoneString())) : false
152         inRange3 = enableTimeFilter3 ? not na(time(timeframe.period, timeFilter3Session, getTimezoneString())) : false
153         inRange1 or inRange2 or inRange3
154
155 // =====
156 // BASE INDICATORS

```

```

157 // =====
158 ema25 = ta.ema(close, 25)
159 ema750 = ta.ema(close, 750)
160 smootherATR = ta.ema(ta.tr, 750)
161 upperBand = ema750 + atrMultiplier * smootherATR
162 lowerBand = ema750 - atrMultiplier * smootherATR
163 vwap = ta.vwap(hl2)
164
165 // VIDYA (parametri fissi come nell'originale)
166 vidya_length = 10
167 vidya_momentum = 20
168 vidya_value = vidya_calc(close, vidya_length, vidya_momentum)
169
170 // === BUTTERWORTH FILTER (parametri fissi come nell'originale) ===
171 bfLength = 20
172 bfPoles = 2
173 bfSrc = close
174
175 get2PoleBF(src_, length) =>
176     PI = 2 * math.asin(1)
177     a = math.exp(-math.sqrt(2) * PI / length)
178     b = 2 * a * math.cos(math.sqrt(2) * PI / length)
179     var float bf = na
180     bf := b * nz(bf[1]) - math.pow(a, 2) * nz(bf[2]) + ((1 - b + math.pow(a, 2)) / 4) * (src_ + 2 * nz(src_[1]) + nz(src_[2]))
181     bf
182
183 get3PoleBF(src_, length) =>
184     PI = 2 * math.asin(1)
185     a = math.exp(-PI / length)
186     b = 2 * a * math.cos(1.738 * PI / length)
187     c = math.pow(a, 2)
188     var float bf = na
189     bf := (b + c) * nz(bf[1]) - (c + b * c) * nz(bf[2]) + math.pow(c, 2) * nz(bf[3]) + ((1 - b + c) * (1 - c) / 8) * (src_ + 3 * nz(src_[1]) + 3 *
nz(src_[2]) + nz(src_[3]))
190     bf
191
192 bf = bfPoles == 2 ? get2PoleBF(bfSrc, bfLength) : get3PoleBF(bfSrc, bfLength)
193
194 // === SWEEP MASTER FILTER ===
195 var int lastSweepBar = na
196 passesSweepMasterFilter() =>
197     not enableSweepMasterFilter or (not na(lastSweepBar) and (bar_index - lastSweepBar) <= sweepMasterFilterBars)
198
199 // =====
200 // LIQUIDITY SWEEP (logica invariata - alimenta lo Sweep Master Filter)
201 // =====
202 swingHighSweepTS = ta.pivohigh(high, swingLenSweep_TS, swingLenSweep_TS)
203 swingLowSweepTS = ta.pivotlow(low, swingLenSweep_TS, swingLenSweep_TS)
204 var float lastSHSweepTS = na
205 var float lastSLSweepTS = na
206 var int lastSHBarSweepTS = na
207 var int lastSLBarSweepTS = na
208 if not na(swingHighSweepTS)
209     lastSHSweepTS := swingHighSweepTS
210     lastSHBarSweepTS := bar_index - swingLenSweep_TS
211 if not na(swingLowSweepTS)
212     lastSLSweepTS := swingLowSweepTS
213     lastSLBarSweepTS := bar_index - swingLenSweep_TS
214 bearSweep_TS1 = not na(lastSHSweepTS) and high > lastSHSweepTS and close < lastSHSweepTS
215 bullSweep_TS1 = not na(lastSLSweepTS) and low < lastSLSweepTS and close > lastSLSweepTS
216 bearSweep_TS2 = false
217 bullSweep_TS2 = false
218 if not na(lastSHSweepTS)
219     for i = 1 to sweepLookback_TS
220         if close[i] > lastSHSweepTS and close < lastSHSweepTS
221             bearSweep_TS2 := true
222             break
223 if not na(lastSLSweepTS)
224     for i = 1 to sweepLookback_TS
225         if close[i] < lastSLSweepTS and close > lastSLSweepTS
226             bullSweep_TS2 := true
227             break
228 bullSweep_TS = enableLiqSweep and (bullSweep_TS1 or bullSweep_TS2) and isInTimeRange() and (not sweepEMAFilter_TS or close > ema750) and (not
sweepVIDYAFilter_TS or (not na(vidya_value) and close > vidya_value)) and (not sweepVWAPFilter_TS or close > vwap) and passesVolScoreFilter() and (not
enableATRMode or close <= upperBand)
229 bearSweep_TS = enableLiqSweep and (bearSweep_TS1 or bearSweep_TS2) and isInTimeRange() and (not sweepEMAFilter_TS or close < ema750) and (not
sweepVIDYAFilter_TS or (not na(vidya_value) and close < vidya_value)) and (not sweepVWAPFilter_TS or close < vwap) and passesVolScoreFilter() and (not
enableATRMode or close >= lowerBand)
230 if bullSweep_TS or bearSweep_TS
231     lastSweepBar := bar_index
232
233 // =====
234 // SEGNALI D'INGRESSO (logica Up/Down Triangle invariata)
235 // =====
236 di = (sm - 1.0) / 2.0 + 1.0
237 c1 = 2 / (di + 1.0)
238 c2 = 1 - c1
239 c3 = 3.0 * (cd * cd + cd * cd * cd)
240 c4 = -3.0 * (2.0 * cd * cd + cd + cd * cd * cd)
241 c5 = 3.0 * cd + 1.0 + cd * cd * cd + 3.0 * cd * cd
242 var float i1 = na
243 var float i2 = na
244 var float i3 = na
245 var float i4 = na

```

```

246 var float i5 = na
247 var float i6 = na
248 i1 := c1 * src + c2 * nz(i1[1])
249 i2 := c1 * i1 + c2 * nz(i2[1])
250 i3 := c1 * i2 + c2 * nz(i3[1])
251 i4 := c1 * i3 + c2 * nz(i4[1])
252 i5 := c1 * i4 + c2 * nz(i5[1])
253 i6 := c1 * i5 + c2 * nz(i6[1])
254 bfr = -cd * cd * cd * i6 + c3 * i5 + c4 * i4 + c5 * i3
255 bfrC = bfr > nz(bfr[1]) ? color.green : bfr < nz(bfr[1]) ? color.red : color.blue
256 colorChange = bfrC != bfrC[1]
257
258 stoK = ta.sma(ta.stoch(close, high, low, 10), 3)
259 longRange = close >= ema750 and (not enableATRMode or close <= upperBand) and close > ema25 and stoK > 50
260 shortRange = close <= ema750 and (not enableATRMode or close >= lowerBand) and close < ema25 and stoK < 50
261
262 volumeMA = ta.sma(volume, 20)
263 volumeRatio = volume / volumeMA
264 volumeScaled = math.max(1, math.min(10, math.round(volumeRatio * 5)))
265
266 passesVolumeFilter(vol) =>
267   if not enableVolumeFilter
268     true
269   else
270     if volumeFilterMode == "Singolo"
271       vol == volumeFilterSingle
272     else
273       vol >= volumeFilterMin and vol <= volumeFilterMax
274
275 // INGRESSO LONG (identico all'Up Triangle originale, tutti i filtri mantenuti)
276 upSignal = enableUpSignal and colorChange and bfrC == color.green and longRange and close > open and close > ema750 and volumeScaled > 5 and
isInTimeRange() and passesVolumeFilter(volumeScaled) and (not enableVWAPFilter or close > vwap) and (not enableEMA750MasterFilter or close > ema750) and
(not enableButterworthMasterFilter or close > bf) and passesVolScoreFilter() and passesSweepMasterFilter() and (not enableATRMode or close <= upperBand)
277 // INGRESSO SHORT (identico al Down Triangle originale, tutti i filtri mantenuti)
278 downSignal = enableDownSignal and colorChange and bfrC == color.red and shortRange and close < open and close < ema750 and volumeScaled > 5 and
isInTimeRange() and passesVolumeFilter(volumeScaled) and (not enableVWAPFilter or close < vwap) and (not enableEMA750MasterFilter or close < ema750) and
(not enableButterworthMasterFilter or close < bf) and passesVolScoreFilter() and passesSweepMasterFilter() and (not enableATRMode or close >= lowerBand)
279
280 // =====
281 // SEGNALI D'USCITA (logica Take Profit System invariata)
282 // =====
283 tpRsi = ta.rsi(close, tpRsiLen)
284 tpMacd = ta.ema(close, tpMacdFastLen) - ta.ema(close, tpMacdSlowLen)
285 tpMacdSignal = ta.ema(tpMacd, tpMacdSignalLen)
286 tpMacdCrossUnder = ta.crossunder(tpMacd, tpMacdSignal)
287 tpMacdCrossOver = ta.crossover(tpMacd, tpMacdSignal)
288 tpK = ta.sma(ta.stoch(close, high, low, tpStochKLen), tpStochSmoothK)
289 tpD = ta.sma(tpK, tpStochDLen)
290 tpStochCrossUnder = ta.crossunder(tpK, tpD)
291 tpStochCrossOver = ta.crossover(tpK, tpD)
292 tpBasis = ta.sma(close, tpBBLen)
293 tpDev = tpBBMult * ta.stdev(close, tpBBLen)
294 tpUpper = tpBasis + tpDev
295 tpLower = tpBasis - tpDev
296 [, tpSTDir] = ta.supertrend(tpSTAtMult, tpSTAtLen)
297 tpCCI = ta.cci(close, tpCCILen)
298 tpEMAFast = ta.ema(close, tpEMAFastLen)
299 tpEMASlow = ta.ema(close, tpEMASlowLen)
300 tpEmaCrossBull = ta.crossover(tpEMAFast, tpEMASlow)
301 tpEmaCrossBear = ta.crossunder(tpEMAFast, tpEMASlow)
302 tpPriceBreakoutBull = close > close[tpLookBackBars]
303 tpPriceBreakoutBear = close < close[tpLookBackBars]
304
305 tpRsiBull = useTPRSI and (tpRsi > tpRsiOB)
306 tpMacdBull = useTPMACD and tpMacdCrossUnder
307 tpStochBull = useTPStoch and tpStochCrossUnder and tpK > 80
308 tpBBBull = useTPBB and close > tpUpper
309 tpSTBull = useTPSTrend and tpSTDir == -1
310 tpCCIBull = useTPCCI and (tpCCI > tpCCIOB)
311 tpEmaBull = useTPEMA and tpEmaCrossBull
312 tpPriceActionBull = useTPPriceAction and tpPriceBreakoutBull
313 tpRsiBear = useTPRSI and (tpRsi < tpRsiIOS)
314 tpMacdBear = useTPMACD and tpMacdCrossOver
315 tpStochBear = useTPStoch and tpStochCrossOver and tpK < 20
316 tpBBBear = useTPBB and close < tpLower
317 tpSTBear = useTPSTrend and tpSTDir == 1
318 tpCCIBear = useTPCCI and (tpCCI < tpCCIOS)
319 tpEmaBear = useTPEMA and tpEmaCrossBear
320 tpPriceActionBear = useTPPriceAction and tpPriceBreakoutBear
321 tpBullCount = (tpRsiBull ? 1 : 0) + (tpMacdBull ? 1 : 0) + (tpStochBull ? 1 : 0) + (tpBBBull ? 1 : 0) + (tpSTBull ? 1 : 0) + (tpCCIBull ? 1 : 0) +
(tpEmaBull ? 1 : 0) + (tpPriceActionBull ? 1 : 0)
322 tpBearCount = (tpRsiBear ? 1 : 0) + (tpMacdBear ? 1 : 0) + (tpStochBear ? 1 : 0) + (tpBBBear ? 1 : 0) + (tpSTBear ? 1 : 0) + (tpCCIBear ? 1 : 0) +
(tpEmaBear ? 1 : 0) + (tpPriceActionBear ? 1 : 0)
323 tpTimeFilterPassed = not applyTPTimeFilter or isInTimeRange()
324 tpVWAPFilterBull = not applyTPVWAPFilter or close > vwap
325 tpVWAPFilterBear = not applyTPVWAPFilter or close < vwap
326
327 // USCITA LONG (Take Profit bullish = chiusura dei long)
328 tpBullSignal = enableTakeProfit and (tpBullCount >= minTPConfirmations) and (tpBullCount > tpBearCount) and tpTimeFilterPassed and tpVWAPFilterBull
329 // USCITA SHORT (Take Profit bearish = chiusura degli short)
330 tpBearSignal = enableTakeProfit and not tpBullSignal and (tpBearCount >= minTPConfirmations) and (tpBearCount > tpBullCount) and tpTimeFilterPassed
and tpVWAPFilterBear
331
332 // =====

```

```

333 // VISUALIZZAZIONE - SOLO I 4 PUNTI RICHIESTI
334 // =====
335 plotshape(upSignal, title = 'Ingresso Long ▲', location = location.belowbar, color = color.new(color.green, 0), style = shape.triangleup, size =
size.small)
336 plotshape(downSignal, title = 'Ingresso Short ▼', location = location.abovebar, color = color.new(color.red, 0), style = shape.triangledown, size =
size.small)
337 plotshape(tpBullSignal, title = 'Uscita Long x', location = location.abovebar, color = color.new(color.green, 0), style = shape.xcross, size =
size.small)
338 plotshape(tpBearSignal, title = 'Uscita Short x', location = location.belowbar, color = color.new(color.red, 0), style = shape.xcross, size =
size.small)
339
340 // =====
341 // ALERTS (invariati per i 4 segnali)
342 // =====
343 alertcondition(upSignal, title = 'Ingresso Long ▲', message = '▲ LONG ENTRY')
344 alertcondition(downSignal, title = 'Ingresso Short ▼', message = '▼ SHORT ENTRY')
345 alertcondition(tpBullSignal, title = 'Uscita Long x', message = 'x LONG EXIT (Take Profit)')
346 alertcondition(tpBearSignal, title = 'Uscita Short x', message = 'x SHORT EXIT (Take Profit)')
347 if upSignal and enableEntryLongAlert
348     alert('▲ LONG ENTRY - VOL: ' + str.toString(volumeScaled) + ' at ' + str.toString(close, format.mintick), alert.freq_once_per_bar_close)
349 if downSignal and enableEntryShortAlert
350     alert('▼ SHORT ENTRY - VOL: ' + str.toString(volumeScaled) + ' at ' + str.toString(close, format.mintick), alert.freq_once_per_bar_close)
351 if tpBullSignal and enableExitLongAlert
352     alert('x LONG EXIT (Take Profit) at ' + str.toString(close, format.mintick), alert.freq_once_per_bar_close)
353 if tpBearSignal and enableExitShortAlert
354     alert('x SHORT EXIT (Take Profit) at ' + str.toString(close, format.mintick), alert.freq_once_per_bar_close)
355

```